TEEプロビジョニングによる認可トークン管理方式の検討

磯部 光平[1,a)]　国井 裕樹[1]

概要：IoT やオンラインサービスの API に対するアクセス制御には、アクセストークンをクレデンシャルとして用いる OAuth が広く使われている。クライアントデバイス上のアクセストークンを安全に管理するには、ハードウェアベースの保護機構を用いることが望ましい。そこで、本稿では、Trusted Execution Environment（TEE）のリモートプロビジョニング機構を用いた OAuth クレデンシャルのプロビジョニング方式を提案する。提案方式では OAuth のアクセストークンやリフレッシュトークンといったクレデンシャルを TEE へのプロビジョニング対象として扱い、その保存と更新を TEE の管理下に置く。これにより OAuth クレデンシャルのプロビジョニング・更新と TEE のリモートアテステーションやリモート管理を連携させる。また提案方式について、クレデンシャルの漏洩に対する安全性と既存の OAuth コンポーネントへの実装上の影響を検討した。その結果、OAuth Client と TEE との連携を担う Helper の構成がアクセストークンの保護水準に影響することを示した。

キーワード：OAuth, Trusted Execution Environment

A Study on Authorization Token Management through TEE Provisioning

Kohei Isobe[1,a)]　Hiroki Kunii[1]

Abstract: OAuth is widely used for API access control in IoT and online services, with access tokens serving as credentials. Hardware-based protection mechanisms are desirable for securely managing access tokens on client devices. This paper proposes an OAuth credential provisioning method that uses a remote provisioning mechanism for a Trusted Execution Environment (TEE). The proposed method treats OAuth credentials, including access tokens and refresh tokens, as provisioning targets for the TEE and places their storage and renewal under the control of the TEE. This approach coordinates the provisioning and renewal of OAuth credentials with remote attestation and remote management of the TEE. We also analyze the security of the proposed method against credential leakage and its implementation impact on existing OAuth components. The results show that the level of protection provided for access tokens during resource access depends on the architecture of the Helper component that coordinates interactions between the OAuth Client and the TEE.

Keywords: OAuth, Trusted Execution Environment

1. はじめに

IoT デバイスやスマートデバイスと、データを収集・配信するサーバとの間では、Web API が広く利用されている。Web API に対するアクセス制御を実現する仕組みとして OAuth[1] がある。OAuth では、API サーバにアクセスするための認可情報をアクセストークンとしてクライアントアプリケーションへ発行し、利用可能なリソースや権限をアクセストークンに対応付けることでアクセス制御を実現する。アクセストークンの漏洩は認可されていないエンティティによるリソースアクセスを可能とし、アクセス制御を担う認可サーバやアクセスを受ける API サーバ、リソースオーナーにとって重大なリスクとなる [2]。このリ

[1] セコム株式会社 IS 研究所
Intelligent Systems Laboratory, SECOM CO.,LTD.
[a)] ko-isobe@secom.co.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

スクを軽減するため、OAuth ではアクセストークンを短期間で失効させるとともに、リフレッシュトークンと呼ばれる長期的なクレデンシャルを用いてアクセストークンの更新を行う仕組みを採用している。

アクセストークンやリフレッシュトークンのようなクレデンシャルの安全な運用には、クレデンシャルを保持・利用するデバイスやアプリケーションの信頼性が重要である。デバイスやその実行環境の信頼性を確保する技術として、ハードウェア機構により実行環境を分離する Trusted Execution Environment（TEE）やリモートアテステーション、リモートプロビジョニングに関する研究や標準化が進められている。これらの技術は、デバイス上で機密データや処理を保護するだけでなく、ネットワークを介してデバイスや実行環境の状態を確認し、その信頼性を検証・維持する仕組みを提供する。クレデンシャルによるアクセス制御を行う API サーバや認可サーバの運用者にとっても、デバイスが信頼できる仕組みや状態を確保することは、クレデンシャルの安全な運用にとって重要な要素である。

クレデンシャルは、それを利用するアプリケーション単独で管理するのではなく、TEE のようなハードウェアベースの信頼できる環境下にプロビジョニングすることがクレデンシャルの運用の観点からも望ましい。この構成では、クレデンシャルの漏洩リスクの低減が期待できるものの、クレデンシャルに加え、デバイスやその信頼性の管理も求められる。OAuth によるクレデンシャルの展開や更新と、TEE やアテステーションを用いたデバイス管理は、それぞれ独立した機構として整備されてきた。そのため、Web API などのシステム設計においては、クレデンシャルの展開とデバイスの管理をそれぞれ設計・運用する必要があり、開発者にとって大きな負担となる。また、TEE はハードウェアによって実装されるために、デバイスの種類などで利用可能な機能や保証水準が異なるため、特に Web API の提供者にとって、両者を連携させたシステム設計の障壁となっている。

本研究では OAuth クレデンシャルの展開や更新を TEE 等のハードウェアベースの保護機構と連携させる際の課題に着目する。特に、クレデンシャルの発行・更新といったイベントと、デバイスおよび TEE の状態確認・更新といったデバイスの管理を個別に扱うのではなく、両者を一体的に扱うことで、開発者や提供者が OAuth クレデンシャルの展開と TEE を持つデバイスの管理を個別に設計・運用する負担の軽減を目指す。具体的には、OAuth クレデンシャルを TEE へのプロビジョニング対象としてとらえ、TEE のリモート管理・プロビジョニング仕様と OAuth を連携させる方式を提案する。著者らは SCIS2023 において同種の検討を行っており [3]、要件や方式の洗い出しを行った。本稿ではこの検討も踏まえ、提案方式の安全性、適用可能性について考察する。

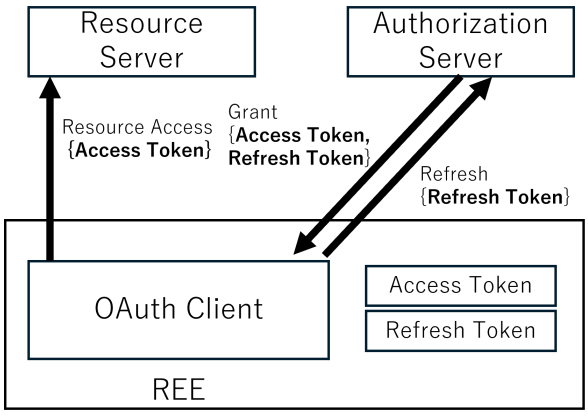


図 1 OAuth におけるアクセストークンの取得・リソースアクセスの概要

Fig. 1 Overview of Grant and Resource Access in OAuth

以降、2 章では OAuth によるアクセス制御の概略とハードウェアベースのセキュリティ機構である TEE の管理についてそれぞれ述べる。3 章では、これらを組み合わせた構成例を示す。4 章では、3 章で示した構成に対して安全性および実装容易性の観点から考察を行う。5 章で関連研究や標準について述べ、6 章で本稿をまとめる。

2. 背景技術

2.1 OAuth

OAuth は、認可サーバ（Authorization Server; AS）が発行するアクセストークンを用いて、API などのリソースを持つリソースサーバ（Resource Server; RS）へのアクセスを制御する認可フレームワークである。OAuth の概観を図 1 に示す。API を利用するクライアント（OAuth Client）は認可サーバからアクセストークンを取得し、アクセストークンをリソースサーバに提示してアクセスする。アクセストークンの取得は Grant Flow と呼ばれる手順によって行われ、認可サーバはリソースオーナーによる認可を確認した後、OAuth Client にアクセストークンを発行する。アクセストークンは一定期間で失効することを前提としており、認可サーバはアクセストークンとともに新たなアクセストークンを取得するためのリフレッシュトークンを発行できる。OAuth Client はリフレッシュトークンを利用することで、再度リソースオーナーの認可を得ることなく新たなアクセストークンを取得できる。また、リソースアクセスが不要になった際には、アクセストークンやリフレッシュトークンを失効させることで、OAuth Client はリソースアクセスが拒否される。このように OAuth では、トークンの発行、配布および更新によってアクセス制御に用いるクレデンシャル管理を実現している。

本稿では IoT デバイスでの利用を前提とした Grant Flow である、Device Authorization Grant Flow[4] を OAuth クレデンシャルのプロビジョニング方式として想定する。

© 2026 Information Processing Society of Japan

Device Authorization Grant Flow は、画面や Web ブラウザなどのユーザインタフェースを持たない OAuth Client 向けに規定された方式である。OAuth Client はアクセストークンの取得をリクエストすると認可サーバから Device Code および User Code、リソースオーナーがアクセスする Verification URL を取得する。OAuth Client は何らかの方法（例：印刷）で User Code と Verification URL をリソースオーナーに与える。リソースオーナーは OAuth Client とは別の端末から Verification URL へアクセスし、User Code を入力した上で認可要求を承認する。一方、OAuth Client は Device Code を用いて認可サーバへ継続的に問い合わせ（ポーリング）を行い、リソースオーナーの承認後にアクセストークンを取得する。

2.2 Trusted Execution Environment

Trusted Execution Environment（TEE）は、通常のアプリケーション実行環境から隔離された実行環境を提供するハードウェアベースのセキュリティ機構である。TEE 内で実行されるプログラムやデータには、通常の実行環境（REE: Rich Execution Environment）からの読取りや改ざんに対する保護が提供される。一般に TEE は、REE と比べ利用可能な計算資源や機能が制約されることから、システム全体を TEE 上で実行するのではなく、機密性や完全性の保護が必要な処理のみを TEE 内で実行する構成が採られる。REE がアプリケーションの大部分を担い、TEE は暗号鍵や機密データの管理、および高い信頼性が要求される処理を担当する。商用実装としては、ARM TrustZone[5] や Intel SGX[6] などが存在する。

2.3 アテステーション・プロビジョニング

アテステーション (Attestation) は、対象となるデバイスや実行環境の属性や状態に関する証拠（Evidence）を生成し、その正当性を検証することによって対象の信頼性を評価する仕組みである。アテステーションによって、検証者は対象となるデバイスや実行環境の識別および状態確認を行うことができる。アテステーションは、同一デバイス内のコンポーネント間で行われる Local Attestation と、ネットワーク越しの検証者との間で行われる Remote Attestation に分類される。特に Remote Attestation は、遠隔でデバイスや実行環境の信頼性を確認する手段として利用される。

デバイスの管理においては、ソフトウェアの導入や更新だけでなく、それらが信頼できる状態で動作していることを確認する必要がある。IETF で標準化が進められている TEEP（Trusted Execution Environment Provisioning）[7] は、TEE 上で動作するアプリケーションや用いられるデータをリモートから管理するための仕様群である。TEEP では、TEE をリモート管理する Trusted Application Manager（TAM）と TEE との間でメッセージを交換し、TEE アプリケーションの導入や更新を実現する。TEEP は TAM と、TEE 搭載デバイス上で動作する TEEP Broker および TEEP Agent によって構成され、TEEP Broker は TEE と TAM との通信を中継し、TEEP Agent は TEE 内で動作する。また、TEEP では TAM から TEE へアテステーションやプロビジョニング要求の情報を収集する `QueryRequest`、Request に対する TEE からの応答である `QueryResponse` および TAM からプロビジョニングを行う `Update` 等の署名付きメッセージを用いて、TEE アプリケーションの状態確認、導入および更新を行う TEEP Protocol というメッセージ仕様の策定が進められている [8] 。

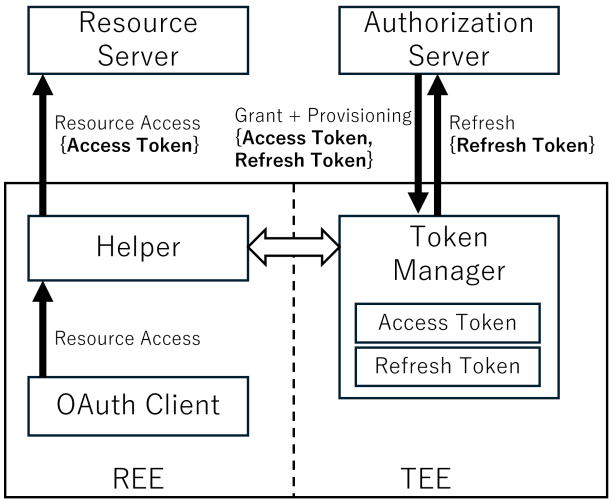


図 2 OAuth クレデンシャルを TEE 管理下にするコンセプト
Fig. 2 Concept of OAuth Credentials under TEE

TEEP では管理対象となる TEE や TEE アプリケーションの状態確認のためにリモートアテステーションを利用する。TAM はアテステーションによって得られる証拠やその検証結果に基づき、管理対象の信頼性を確認したうえでソフトウェアの配信を実施できる。このように、TEE のリモート管理においては、アプリケーションやデータのプロビジョニングや更新とアテステーションによる信頼性確認が密接に関連している。

3. 提案方式

3.1 設計方針と構成

提案方式は、OAuth クレデンシャルのプロビジョニングや更新と、TEE におけるデバイス管理やリモートアテステーションを統合することを目的とする。このコンセプトを図 2 に示す。

提案方式では、OAuth のアクセストークンとリフレッシュトークンを TEE に対するプロビジョニング対象として扱うことで、OAuth のクレデンシャル管理を TEEP を用いた TEE のリモート管理と統合する。また、クレデンシャルの発行および更新時に TEE のアテステーションを

© 2026 Information Processing Society of Japan

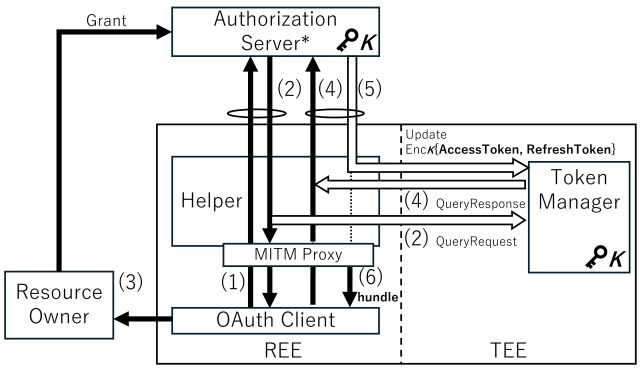


図 3 提案方式によるアクセストークンのプロビジョニング
Fig. 3 Provisioning Access Token in Proposed Method

実施することで、クレデンシャルのプロビジョニングとプロビジョニング先であるデバイスや TEE の信頼性確認を連携させる。

提案方式は、OAuth Client、Authorization Server（AS）、Resource Server（RS）、Helper および TEE 上で動作する Token Manager（TM）によって構成される。

- AS は OAuth の認可サーバのほかに、TEEP における Trusted Application Manager（TAM）としても動作する。
- OAuth Client は REE 上で動作するアプリケーションである。
- Helper は OAuth Client と AS および RS との通信を Machine-in-the-Middle (MITM) 型の HTTP プロキシとして中継する機能と、TEE 内の TM と AS との通信を仲介する TEEP Broker 機能を有する。TM と連携し、REE で動作する。
- TM は TEE 上で動作し、アクセストークンおよびリフレッシュトークンの管理と TEEP Agent の機能を有しアテステーションの証拠生成を担う。

AS と TM は事前に暗号化のための共有鍵 K を保持しているものとし、トークンのプロビジョニングに利用する。

提案方式では、AS が発行した実際のアクセストークンおよびリフレッシュトークンは TM のみが保持する。OAuth Client は実際のアクセストークンの代わりに対応するハンドルを保持する。

3.2 提案方式

3.2.1 アクセストークン・リフレッシュトークンのプロビジョニング

Device Authorization Grant Flow を利用してアクセストークンとリフレッシュトークンを取得する。プロビジョニングの流れを図 3 に示す。OAuth Client は AS に対して Device Authorization Request を送信し **(1)**、Device Code、User Code および Verification URL を取得する。このとき AS は TM に対する `QueryRequest` メッセージを重畳する。Helper は OAuth Client に対しては Grant Flow に基づくパラメータを渡し、`QueryRequest` は TM に送信する **(2)**。OAuth Client は User Code と Verification URL をリソースオーナーに提示し **(3)**、リソースオーナーは別端末から Verification URL へアクセスし、User Code を入力したうえで認可要求を承認する。OAuth Client は認可要求が承認されるまで、AS に対して Device Code を含めてポーリングを行う **(4)**。一方、TM は Helper から得た `QueryRequest` に対する `QueryResponse` を生成し、Helper へ応答する。これには TEE の状態に関するアテステーションの証拠を含む。AS (TAM) に対する `QueryResponse` の送信は、OAuth Client が行うポーリングに重畳する。すなわち、Helper は OAuth Client がポーリング通信を行う際に、TM から入手した `QueryResponse` を含めたうえで AS に送信する。

AS は認可要求のリソースオーナーの承認および `QueryResponse` に含まれる証拠を検証した後、アクセストークンとリフレッシュトークンを生成する。生成されたアクセストークンとリフレッシュトークンは、TM に対するプロビジョニングとして、`Update` メッセージに含めたうえで、TM との間で共有する鍵 K によって暗号化を行う。AS は、暗号化された `Update` を TM へ送信する **(5)**。TM は受信した `Update` を復号し、アクセストークンとリフレッシュトークンを保存する。ポーリングを要求してきた OAuth Client に対してはアクセストークンに対応するハンドルを返送する **(6)**。

3.2.2 リソースアクセス

OAuth Client はハンドルを用いてリソースアクセスを行う。OAuth Client は、Helper を HTTPS プロキシとして通信を開始する。そのため、アクセス要求は Helper へ送信される。Helper は TM に対して実際のアクセストークンを要求し、TM はハンドルに対応するアクセストークンを Helper へ返送する。Helper は OAuth Client から受信した HTTP リクエスト中のハンドルを、TM から取得した実際のアクセストークンへ置換する。Helper は置換後のリクエストを RS へ転送する。

RS は AS に対してアクセストークンの有効性確認を行う。AS はアクセストークンの検証結果を RS へ返送し、有効なアクセストークンであった場合に RS はリソースアクセスを処理する。処理結果は RS から Helper へ返送され、最終的に Helper から OAuth Client へ返送される。

通常の OAuth Client と RS 間のリソースアクセスとの差分は、MITM Proxy として振る舞う Helper が介在することだけであり、OAuth Client や RS は OAuth の標準仕様に則り、リソースアクセスを行うことができる。

3.2.3 リフレッシュトークンを用いたアクセストークンの更新

アクセストークンの有効期限が近づくと、TM は保持し

© 2026 Information Processing Society of Japan

ているリフレッシュトークンを利用してアクセストークンの更新を開始する。TM は Helper を介して AS へ更新要求を送信する。AS は TM の状態確認を行うため、TEEP の `QueryRequest` を送信する。これに対して TM は、証拠を含む `QueryResponse` を返送する。AS は証拠など得られた応答を検証し、TM が期待する状態で動作していることを確認する。検証に成功した場合、AS は新しいアクセストークンを生成する。必要に応じて、新たなリフレッシュトークンも生成する。AS は生成したトークンを `Update` に含めて共有鍵 K で暗号化したうえで TM へ送信する。TM は受信した `Update` を復号し、保持しているアクセストークンおよびリフレッシュトークンを更新する。更新後も、OAuth Client が保持するハンドルは変更されない。以後のリソースアクセスでは、TM が更新後のアクセストークンを利用する。

4. 考察

この章では、提案方式を、クレデンシャル漏洩に対する安全性と、既存 OAuth コンポーネントへの実装上の影響の二つの観点から考察する。

4.1 クレデンシャル漏洩に対する安全性

デバイスからアクセストークンが漏洩した場合、攻撃者は正規のデバイスや OAuth Client になりすまして RS へアクセスできる。リフレッシュトークンの漏洩は攻撃者に AS からの継続的なアクセストークンの取得を可能とする。本稿では、これらに対応しデバイス上に保存または処理されるクレデンシャルの漏洩を脅威として想定する。漏洩元としては、デバイスのストレージやメモリを考慮する。一方で、デバイスと AS や RS を結ぶ通信路上の盗聴については OAuth が前提とする Transport Layer Security (TLS) によって保護される [1] ものとする。また TEE 実装に対する攻撃やサイドチャネル攻撃等による TEE 内部情報の取得など TEE そのものに対する攻撃は検討の対象外とする。

アクセストークンの漏洩について、提案方式では TEE 上で動作する TM がアクセストークンをストレージに保存する。OAuth Client や Helper、その他の REE 上のアプリケーションは実際のアクセストークンを保持せず、直接参照することもできない。REE やそのストレージが侵害されても、TM が保存したアクセストークンが漏洩することはない。

リソースアクセスなどストレージに保存されたアクセストークンを利用する際は、Helper と TM が協調してアクセストークンを利用する。そのため、アクセストークンが Helper が実行される REE へ出力される実装を許容した場合にはメモリからの漏洩や OAuth Client が侵害された場合において保護効果が低下する。

リフレッシュトークンの漏洩については、Helper の構成に依存せず TEE 上で動作する TM と AS でのみ取り扱われるため、REE 上のストレージやメモリ、また OAuth Client や Helper への攻撃を受けても安全であるといえる。なお、アクセストークンの発行および更新時には、TEE によるアテステーション情報が AS に送信される。AS は、デバイスが期待するデバイスの状態・信頼性を満たしていることを確認したうえで、アクセストークンを発行・更新することができる。これによりデバイスにおけるクレデンシャルの保護や管理の程度を把握しながら運用できるため、展開先のリスクを踏まえたクレデンシャルの運用が可能と言える。一方で、アテステーションによってデバイス全体の安全性が保証されるわけではなく、検証対象外のソフトウェアや実行環境に起因する攻撃は残存する可能性がある。

クレデンシャル漏洩に対する安全性を表 1 に示す。以上より、提案方式は構成によってトークン漏洩そのものを完全に防止するものではないものの、リフレッシュトークンを含むクレデンシャル保護と TEE の管理機構を統合した構成と位置付けられる。

4.2 標準仕様からの変化

OAuth クレデンシャルの管理とハードウェアの管理の両立に伴う負担を評価するにあたり、本稿ではその一側面として、OAuth 標準に準拠する既存コンポーネントに必要となる変更を検討する。OAuth にかかわる OAuth Client、RS、 AS ごとに差分を検討する。

OAuth Client は本提案では Helper を介して RS と通信を行う。Helper は MITM Proxy として振る舞うため、OAuth Client は MITM Proxy を介して通信を行うよう対応しなければならない。ただ MITM Proxy が介在する箇所は OAuth が利用する HTTP プロトコルのレイヤーであり、OAuth プロトコルそのものに対しては改変を行っていない。そのため、既存の OAuth Client の標準仕様から変更せず対応できると考えられる。OAuth 仕様やそれに対応する実装を変更する必要がないため、OAuth Client の機能が組み込まれているソフトウェアに対しても改変を加えず、本手法は適用可能であると考えられる。

RS は、Helper が仲介し送信するリソースアクセスは通常の OAuth のリソースアクセスと同様にアクセストークンを含んでおり OAuth 標準に基づき処理が可能である。本提案に基づく OAuth Client と通常の OAuth Client が混在しても RS は区別なく処理することができる。

AS は、TAM の機能を有するため構成として TAM に関連するコンポーネントや処理を追加しなければならない。OAuth 標準仕様からの変更がある箇所は、アクセストークンやリフレッシュトークンの発行や更新に関わる通信内容が該当し、この箇所を TEEP メッセージに置き換える対応をしなければならない。

© 2026 Information Processing Society of Japan

表 1　クレデンシャル漏洩に対する安全性
Table 1　Security for Credential Leakage

イベント	アクセストークン	リフレッシュトークン	保護の仕組み
プロビジョニング	✓	✓	TM と AS 間の暗号化鍵を使い展開される
保存	✓	✓	TEE のみアクセスできるように保存
リソースアクセス	△	-	デバイスと RS 間は TLS で保護されるものの、Helper と OAuth Client 間の連携方法によっては REE 上のメモリに出力される場合がある。
更新	✓	✓	TM と AS 間の暗号化鍵を使い更新される

5. 関連研究・標準

5.1 関連研究

TEE を用いてクレデンシャル管理を行う研究として、SafeKeeper[9] や、DelegatTEE[10] は、Intel SGX を用いて Web サービスのログインに用いるパスワードをエンクレーブに格納したうえで、ウェブサイトへのログインやユーザ間の共有を実現している。MISO[11] は、OAuth を拡張した OpenID Connect (OIDC) を利用したシングルサインオンにおいて、ID プロバイダがログイン先 (Relying Party) を追跡可能という課題に対し、SGX 上で動作する ID ブローカーを提案しており、OIDC のアクセストークンを SGX エンクレーブで保持・管理する。MA-TEECM[12] は、TrustZone を搭載した携帯端末に保存されたクレデンシャルを信頼できる第三者の仲介なしに別の端末へ移行する手法を提案している。

TEE の利用状況について調べた Niu らの研究 [13] では、調査対象となった TEE を利用する 241 のオープンソースプロジェクトのうち、71 のプロジェクトが IoT デバイスのセキュリティ用途で TEE を利用しており、最も多い用途だったと報告している。デバイスの管理や制御を目的とした TEE やアテステーションを用いた研究として、Julku らの研究 [14] では、OAuth 認可フローへアテステーションを統合し、ユーザが認可する際にデバイスの信頼性も確認可能とする方式を提案している。クレデンシャルを展開とデバイス信頼性管理が結び付けられているものの、アテステーション結果を認可時に確認するのみであり、デバイスにおけるクレデンシャルの管理は、OAuth Client のままである。また SENG[15] はアプリケーションを SGX Enclave に格納して実行することで、Enclave の制御を通じた、アプリケーションに紐づいたファイヤーウォール機構を実現しており、デバイス管理の一つととらえることができる。しかしながらアプリケーションを Enclave に格納する方式のため、アプリケーションの動作に制約がありセンサなど外部入出力などを用いるアプリケーションには適用できない。

以上より、既存研究では「クレデンシャル保護と TEE の統合」と「デバイスの管理や制御と TEE やアテステーションの統合」は検討されていることが分かる。

5.2 関連標準

OAuth に関連する仕様群が規定する対策との比較を行う。OAuth 仕様においても、アクセストークンやリフレッシュトークンの漏洩やそれに伴う不正なリソースアクセスは脅威としてとらえられている。この脅威への対策としてアクセストークンの短寿命化やアクセス権限範囲の最小化などと共に、トークンを特定の送信者に限定する方式を Sender-constrained Token として定め、これを実現する技術仕様を策定している。Sender-constrained Token の仕様に該当する標準としては、DPoP (Dynamic Proof of Possession)[16] と Mutual TLS[17] がある。それぞれ OAuth Client が認証用の公開鍵ペアを作成する。DPoP の場合はトークン使用時に OAuth Client が署名を付加する。Mutual TLS では、RS との通信に認証用鍵を用いた Mutual TLS の通信路を確立し、RS は OAuth Client が TLS での認証に用いた公開鍵とアクセストークンに紐づく公開鍵が一致することを確認する。いずれもアクセストークン単独の漏洩には耐性を有している。これら方式の利用には、OAuth Client での鍵生成や管理が求められる。認証用の秘密鍵の漏洩や OAuth Client を介した不正な利用は、トークンの不正利用につながるため保護が求められるが、OAuth の仕様においては認証鍵の保護策は実装依存とされている。

提案方式では、アクセストークンやリフレッシュトークンは、TEE と AS が合意した暗号化鍵 K で暗号化された状態で TEE にプロビジョニングされる。利用時においては、アクセストークンについては OAuth Client と Helper のいずれも認証や署名は不要であり、それに伴う鍵管理も不要である。リフレッシュトークンの更新や使用については、TEEP に基づき署名や暗号化を TEE で実施して行うが、暗号鍵は TEE 上で保護されるため、一度 AS と TEE が鍵合意できれば TEE や AS が安全である限りは、鍵更新は不要である。

6. おわりに

本稿では、OAuth クレデンシャルのプロビジョニングと

© 2026 Information Processing Society of Japan

TEE の管理の統合について検討し、OAuth クレデンシャルを TEE に対してプロビジョニングを行う構成を提案した。また、提案方式について OAuth クレデンシャルの漏洩に対する安全性の検討、先行研究および関連標準との比較を行った。その結果、OAuth クレデンシャルは認可サーバと TEE 間で機密性が確保されるものの、そのうち REE で動作する OAuth Client が用いるアクセストークンについては、TEE との連携方法により安全性が変化しうることを示した。既存研究では OAuth とアテステーションの統合、あるいは TEE によるクレデンシャル保護はそれぞれ検討されている一方で、OAuth におけるクレデンシャル管理と TEE におけるデバイス管理を統合的に扱う検討は限定的であることを示した。今後の課題としては、提案方式の実装および評価を通じて、統合による運用上の有効性や TEE やアテステーションの実装差異を加味して安全性・性能を検証することが挙げられる。

参考文献

[1] Hardt, D.: The OAuth 2.0 Authorization Framework, RFC 6749 (2012).

[2] Farooqi, S., Zaffar, F., Leontiadis, N. and Shafiq, Z.: Measuring and Mitigating Oauth Access Token Abuse by Collusion Networks, *Proceedings of the 2017 Internet Measurement Conference*, IMC '17, New York, NY, USA, Association for Computing Machinery, p. 355–368 (online), DOI: 10.1145/3131365.3131404 (2017).

[3] 磯部光平，八代理紗，国井裕樹：トークン型クレデンシャルに対する IoT デバイス向けハードウェアベース保護機構の適用に関する考察，暗号と情報セキュリティシンポジウム (scis) 2023 (2023).

[4] Denniss, W., Bradley, J., Jones, M. and Tschofenig, H.: OAuth 2.0 Device Authorization Grant, RFC 8628 (2019).

[5] ARM Limited.: ARM Security Technology Building a Secure System using TrustZone Technology (2009).

[6] Costan, V. and Devadas, S.: Intel SGX Explained, Cryptology ePrint Archive, Paper 2016/086 (2016).

[7] Pei, M., Tschofenig, H., Thaler, D. and Wheeler, D.: Trusted Execution Environment Provisioning (TEEP) Architecture, RFC 9397 (2023).

[8] Tschofenig, H., Pei, M., Wheeler, D., Thaler, D. and Tsukamoto, A.: Trusted Execution Environment Provisioning (TEEP) Protocol, Internet-Draft draft-ietf-teep-protocol-26, Internet Engineering Task Force (2026).

[9] Krawiecka, K., Kurnikov, A., Paverd, A., Mannan, M. and Asokan, N.: SafeKeeper: Protecting Web Passwords using Trusted Execution Environments, *Proceedings of the 2018 World Wide Web Conference*, WWW '18, Republic and Canton of Geneva, CHE, International World Wide Web Conferences Steering Committee, p. 349–358 (online), DOI: 10.1145/3178876.3186101 (2018).

[10] Matetic, S., Schneider, M., Miller, A., Juels, A. and Capkun, S.: DelegaTEE: Brokered Delegation Using Trusted Execution Environments, *27th USENIX Security Symposium (USENIX Security 18)*, Baltimore, MD, USENIX Association, pp. 1387–1403 (online), available from ⟨https://www.usenix.org/conference/usenixsecurity18/presentation/matetic⟩ (2018).

[11] Xu, R., Yang, S., Zhang, F. and Fang, Z.: MISO: Legacy-compatible Privacy-preserving Single Sign-on using Trusted Execution Environments, *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, pp. 352–372 (online), DOI: 10.1109/EuroSP57164.2023.00029 (2023).

[12] Wang, Z., Wang, L. and Yan, H.: MA-TEECM: Mutual Anonymous Authentication-Based Credential Migration Technology for Mobile Trusted Execution Environments, *IEEE Access*, Vol. 11, pp. 3680–3690 (online), DOI: 10.1109/ACCESS.2023.3235372 (2023).

[13] Niu, Y., Shi, J., Han, R., Liu, Y., Ma, C., Lyu, Y. and Lo, D.: What You Trust is Insecure: Demystifying How Developers (Mis)Use Trusted Execution Environments in Practice, *2026 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 228–239 (online), DOI: 10.1109/SANER67736.2026.00034 (2026).

[14] Julku, J., Suomalainen, J. and Kylänpää, M.: Delegated Device Attestation for IoT, *2021 8th International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*, pp. 1–8 (online), DOI: 10.1109/IOTSMS53705.2021.9704959 (2021).

[15] Schwarz, F. and Rossow, C.: SENG, the SGX-Enforcing Network Gateway: Authorizing Communication from Shielded Clients, *29th USENIX Security Symposium (USENIX Security 20)*, USENIX Association, pp. 753–770 (online), available from ⟨https://www.usenix.org/conference/usenixsecurity20/presentation/schwarz⟩ (2020).

[16] Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M. and Waite, D.: OAuth 2.0 Demonstrating Proof of Possession (DPoP), RFC 9449 (2023).

[17] Campbell, B., Bradley, J., Sakimura, N. and Lodderstedt, T.: OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens, RFC 8705 (2020).

© 2026 Information Processing Society of Japan